SALESFORCE INTEGRATION ANALYSIS AND PLANNING

Week 3 Task Report

Prepared by	Nidhrabingi Kavitha
Task	Salesforce Integration Analysis and Planning
Systems analyzed	ERP platform and Marketing Automation platform
Deliverables	Integration report and technical best-practices guide

A practical integration planning simulation covering architecture, interfaces, data flows, security, risks, implementation, testing, monitoring, and maintenance.

1. Executive Summary

This report proposes an integration approach connecting Salesforce with two common enterprise systems: an Enterprise Resource Planning (ERP) platform and a Marketing Automation platform. ERP integration supports account/customer alignment, product and order visibility, and fulfillment status updates. Marketing integration supports campaign engagement, lead capture, segmentation, lead scoring, and campaign outcome reporting. API-based integration is recommended, with middleware where transformation, orchestration, retries, and centralized monitoring are needed. This is a simulated design; no live Salesforce environment has been inspected or connected.

2. Objectives and Scope

The objective is to identify integration touchpoints, explain data exchanged, recommend interfaces and patterns, assess technical and operational risks, and define a step-by-step delivery plan. The scope includes system boundaries, authentication, data mapping, error handling, testing, deployment, monitoring, and ongoing ownership. Exact product capabilities and field mappings must be confirmed with system owners before implementation.

3. Systems and Business Use Cases

A. ERP (e.g., SAP, Oracle ERP, Microsoft Dynamics): exchange approved customer/account identifiers, product and price data, order summaries, fulfillment status, and invoice/payment status. Salesforce should remain the sales activity and opportunity system of record; the ERP should own order fulfillment and financial transactions.

B. Marketing automation (e.g., Salesforce Marketing Cloud or another campaign platform): exchange consented lead/contact attributes, campaign membership, engagement events, lead scores, and campaign attribution. Salesforce can return sales status and outcomes. Consent and communication preferences must be honored in both directions.

4. Proposed Integration Architecture

Reference architecture: Salesforce CRM ↔ HTTPS APIs / events ↔ Middleware or iPaaS ↔ ERP and Marketing Automation. Middleware is recommended for complex mappings, multiple systems, retries, and centralized observability. A direct API integration may suit a smaller, low-complexity use case after security and operational requirements are reviewed.

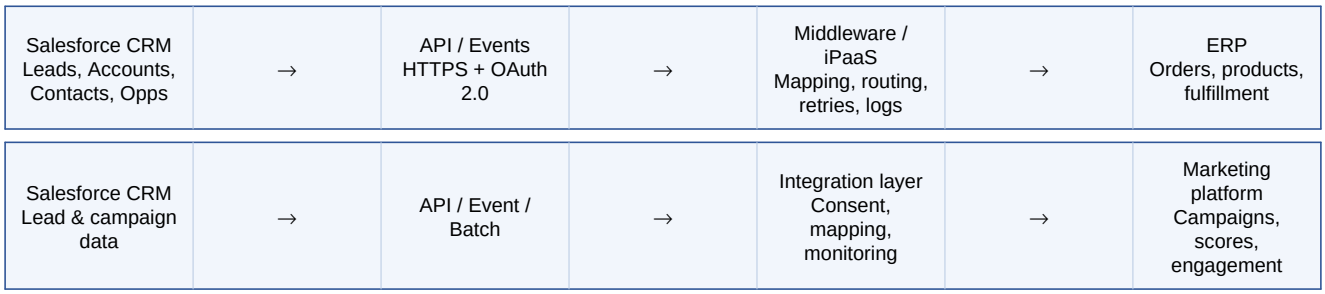

Figure 1. Logical architecture and primary integration touchpoints. Validate licensing, product capabilities, API availability, network constraints, and volumes during discovery.

5. Integration Points and Data Flows

Integration	Direction	Example data	Pattern
Salesforce ↔ ERP	Bi-directional with ownership rules	Customer reference, products, order and fulfillment status	API for lookups; events or scheduled batch for updates
Salesforce ↔ Marketing	Bi-directional	Lead/contact attributes, consent, campaigns, engagement, lead score	Events/API for timely updates; batch for large audiences
Middleware / iPaaS	Between systems	Canonical model, mappings, routing, logs, error records	Orchestration, transformation, retries, error queue

6. Technical Requirements

• **Interfaces:** Salesforce REST API for standard record operations; Bulk API 2.0 for large asynchronous loads; Platform Events or Change Data Capture for suitable event-driven use cases; middleware connectors where available.

• **Authentication:** OAuth 2.0 and HTTPS/TLS, dedicated least-privilege integration identity, approved secrets manager, and periodic credential rotation. For new client integrations, review Salesforce External Client Apps guidance; Salesforce notes restrictions on creating new Connected Apps beginning Spring ’26.

• **Data contracts:** document source/target fields, types, required fields, picklist mappings, time zones, currencies, identifiers, null handling, and ownership rules. Prefer stable external IDs for upsert and deduplication.

• **Reliability:** timeouts, bounded retries with exponential backoff, idempotency, replay/reconciliation where supported, error queues, and alerts. Avoid endless retries for permanent validation or authorization failures.

• **Capacity:** verify org-specific API allocations, concurrency limits, payload constraints, and vendor rate limits. Batch/bulk requests where appropriate and avoid unnecessary polling.

7. Data Ownership and Mapping Rules

• Define one authoritative owner per field/entity: ERP for orders/invoices; Salesforce for sales stage and activities; marketing platform for campaign engagement. Agree separately on consent ownership.

• Use a shared external customer ID; do not match solely on names. Define duplicate handling and merge governance before synchronization.

• Normalize country, state, currency, date/time, phone, and status values. Document transformations and quarantine records that cannot be mapped safely.

• Limit the integration identity by object and field permissions; use appropriate sharing and audit controls.

8. Risk Assessment and Controls

Risk	Impact	Control
Unauthorized access/data exposure	Customer or financial data disclosure	OAuth 2.0, least privilege, TLS, secret vault, audit logs, access reviews
Mismatch/duplicates	Incorrect records or reporting	External IDs, validation, idempotent upserts, reconciliation
Latency/stale data	Outdated order or campaign status	Set latency targets; choose event vs batch based on need; alert on lag
API/vendor limits	Delayed or failed synchronization	Monitor usage, batch requests, queue workloads, backoff
Partial failures/retries	Duplicate or missing updates	Idempotency, error queue, replay and reconciliation
Schema/version changes	Integration failure after updates	Contract tests, versioned mappings, release review
Ownership conflicts	Circular overwrites	Field ownership matrix, source markers, loop prevention

9. Step-by-Step Implementation Plan

Step 1 — Discover — Interview sales, marketing, finance/operations, security, and system owners. Confirm use cases, volumes, latency targets, compliance needs, system-of-record rules, and success criteria.

Step 2 — Design — Select direct API or middleware. Create architecture/sequence diagrams, data dictionary, field ownership matrix, authentication design, retry policy, monitoring plan, and rollback approach.

Step 3 — Prepare environments — Use separate development/test and production environments. Configure integration identity, OAuth settings, permissions, network restrictions where applicable, and secrets storage.

Step 4 — Build — Implement transformations, external-ID matching, validation, idempotency, event/batch scheduling, error routing, and correlation IDs.

Step 5 — Test — Run unit, mapping, contract, integration, negative, security, volume, performance, and recovery tests. Include duplicates, unavailable endpoints, expired credentials, invalid fields, and throttling.

Step 6 — User acceptance — Have business owners verify lead, campaign, account, order, and status scenarios. Reconcile source/target counts and obtain sign-off.

Step 7 — Deploy safely — Use controlled change management, verify production configuration, enable monitoring, start with limited volume, and keep a rollback plan.

Step 8 — Operate — Review failures, latency, API use, reconciliation, and KPIs. Update mappings and runbooks when systems or rules change.

10. Testing, Monitoring, and Maintenance

• Monitor successful/failed records, end-to-end latency, queue depth, retry count, API usage, last successful sync, and reconciliation variance.

• Use correlation IDs and sanitized error details. Never log passwords, tokens, payment details, or unnecessary personal data.

• Alert on sustained failures, authentication errors, dead-letter growth, sync lag beyond target, and approaching API/vendor limits.

• Maintain a runbook with owners, escalation contacts, replay/restart instructions, known errors, recovery steps, and incident communication.

• Schedule access reviews, secret rotation, dependency/version reviews, recovery drills, and mapping audits.

11. Success Measures

• Integration success rate and records reconciled without manual correction

• Median and 95th-percentile synchronization latency against agreed targets

• Duplicate rate and mapping/validation error rate

• Time to detect and resolve incidents

• API consumption compared with allocation

• Business outcomes such as faster lead follow-up and improved order-status visibility

12. Conclusion

Successful integration requires more than an API connection. It needs clear data ownership, secure access, resilient error handling, operational monitoring, and ongoing governance. The ERP and marketing scenarios demonstrate how to choose patterns based on volume and timeliness, use middleware when orchestration is valuable, and validate design through testing and reconciliation. Final design should be approved by system owners after confirming product capabilities, licensing, data classifications, and org-specific limits.

13. References

Salesforce Developers — Integration Patterns and Practices:
https://developer.salesforce.com/docs/atlas.en-us.integration_patterns_and_practices.meta/integ_pat_tempate.htm

Salesforce Developers — OAuth 2.0 and External Client Apps:
https://developer.salesforce.com/docs/platform/api-rest/guide/intro-oauth-and-connected-apps.html

Salesforce Developers — API Request Limits:
https://developer.salesforce.com/docs/platform/salesforce-app-limits-cheatsheet/guide/salesforce-app-limits-platform-api.html

Salesforce Help — Salesforce Connector: https://help.salesforce.com/s/articleView?id=001121996&language;=en_US&type;=1

Note: Proposed planning simulation based on public documentation and common integration practices; not a live production audit.